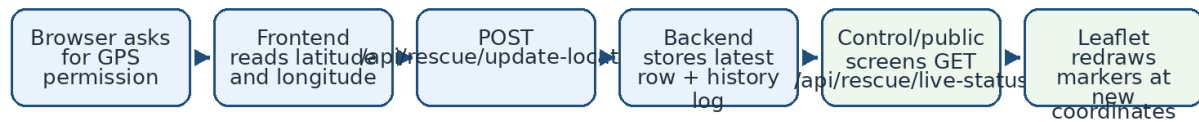


NDRCS Live Location System Explained Properly

Frontend + backend + maps + polling + marker movement

How location flows through NDRCS



This is polling: rescue side keeps sending updates every few seconds, and monitoring screens keep fetching the newest rows every few seconds.

This document is based on the current NDRCS codebase and explains the real approach used there: browser geolocation, REST API calls, MySQL storage of latest position plus history, and repeated polling on monitoring screens.

Your understanding is basically correct: when the browser asks for location permission and you allow it, the frontend gets GPS coordinates, sends them to the backend, and repeated API calling on both sides creates the live-tracking effect.

1. Which map technology NDRCS uses

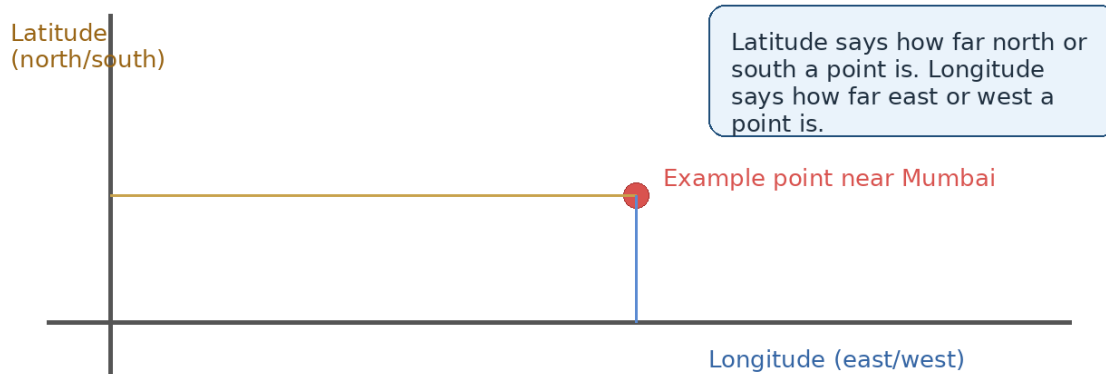
Your project uses Leaflet on the frontend and OpenStreetMap tiles underneath it. Leaflet is the JavaScript library that draws the interactive map, creates markers, supports zoom, layer groups and popups. OpenStreetMap supplies the map tiles using the URL pattern `https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png`. The backend is not a map server; it only stores and returns team and incident coordinates.

- Leaflet appears in the frontend through `L.map`, `L.tileLayer`, `L.marker`, `L.layerGroup` and `L.divIcon`.
- OpenStreetMap is the background road map you see under the markers.
- The frontend centers the map around Mumbai using `[19.0760, 72.8777]`.
- The same mapping approach is used on the public map, control room dashboard and rescue dashboard.

2. Latitude and longitude in plain words

Any point on Earth can be described by two numbers. Latitude tells how far north or south the point is. Longitude tells how far east or west the point is. The browser returns these values through `pos.coords.latitude` and `pos.coords.longitude`. Leaflet then places a marker at `[latitude, longitude]`.

Latitude and longitude in simple words



Important note: In Leaflet, the order is [latitude, longitude]. If those two are swapped, the marker will appear in the wrong place.

3. What happens when the browser asks for location permission

On the rescue side, the code checks whether geolocation is available in the browser. Then it calls `navigator.geolocation.getCurrentPosition(...)`. At that point the browser shows the permission popup. If the user allows access, the callback receives the latest GPS reading. If the user denies access, your code falls back to the default map center and shows an alert.

- Permission is handled by the browser, not by Spring Boot.
- The frontend does not permanently store the browser permission itself; it only receives coordinates when the browser allows them.
- In your rescue dashboard, `currentLat` and `currentLng` are updated from the browser reading.
- After that, the frontend sends those coordinates to the backend using `POST /api/rescue/update-location`.

4. Exact frontend flow in your project

Rescue dashboard flow

The rescue dashboard starts auto tracking immediately after page load. It calls `getCurrentPosition` once, then again every 5 seconds through `setInterval`. Each successful GPS reading updates `currentLat/currentLng`, refreshes the local marker and sends a `POST` request to the backend with `userId`, `latitude`, `longitude`, `locationName`, `statusMessage` and `urgencyLevel`.

Control room flow

The control room dashboard does not fetch GPS from the browser for team tracking. Instead it repeatedly calls the backend endpoints `/api/dashboard/summary`, `/api/incidents` and `/api/rescue/live-status`. It clears the old marker layers and plots fresh markers using the newest coordinates returned by the backend.

Public map flow

The public map follows the same monitoring idea. Every 5 seconds it fetches incidents and rescue-team live status, clears the previous layer groups, filters out resolved incidents and offline teams, and adds new Leaflet markers with popups.

5. Exact backend flow in your project

The backend side is very small but important. The `RescueTeamController` exposes two endpoints under `/api/rescue`. `POST /update-location` passes the request to `RescueTeamService.updateLiveLocation()`. `GET /live-status` returns all rows from the live-status table so the monitoring screens can plot them on the map.

- The service first finds the `AuthUser` using `userId`.
- It checks whether that account role is `RESCUE_TEAM`.
- It finds the linked `RescueTeamProfile`.
- It loads the current `RescueTeamLiveStatus` row for that team, or creates one if none exists.
- It updates `latitude`, `longitude`, `locationName`, `statusMessage`, `urgencyLevel` and `updatedAt`.
- It saves a new `RescueTeamLocationLog` row to keep history of movement.

Why two tables? `RescueTeamLiveStatus` stores only the latest known position for each team. `RescueTeamLocationLog` stores the full history of position updates over time. This is a good design because the map can fetch the latest row quickly while you still preserve a movement history for analysis.

6. What polling means here

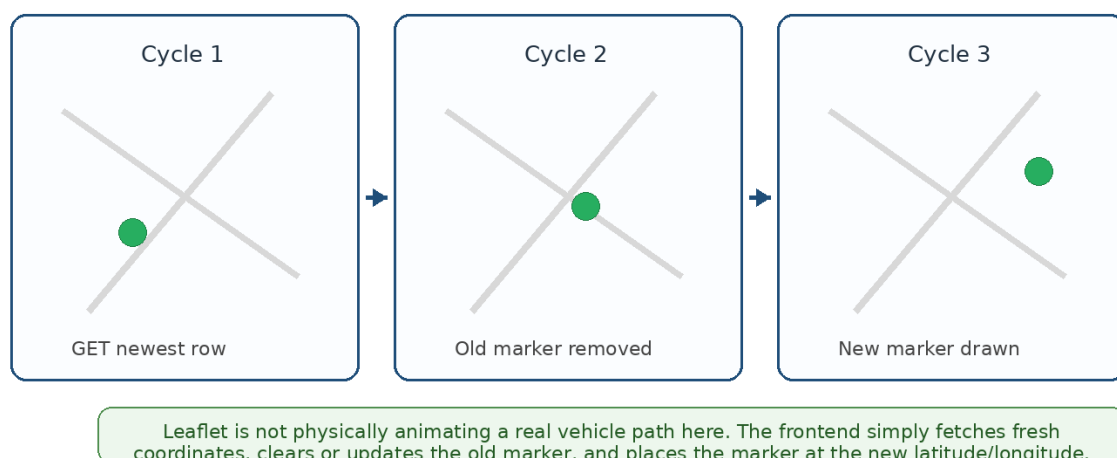
Polling means the app does not wait for the server to push updates. Instead, the client keeps asking again and again after a fixed time gap. In your codebase, the rescue side polls the browser for GPS and POSTs updates. The control room and public map poll the backend and GET the newest rows. This repeated cycle is what creates the live feel without WebSockets.

- Rescue browser loop: every 5 seconds -> get GPS -> POST update-location.
- Monitoring loop: every 5 seconds -> GET live-status -> redraw markers.
- Because both loops repeat, the screen stays fresh enough for a student project and demo.

7. How marker movement happens on the map

The marker is not truly travelling in a continuous animation. What really happens is simpler: the frontend receives a new coordinate pair and either updates the existing marker position or clears and redraws markers from fresh backend data. Since this happens repeatedly, your eyes see motion.

How the map marker appears to move



- On the rescue dashboard, `updateMapCenter()` uses `myMarker.setLatLng([currentLat, currentLng])` when the marker already exists.

- On the control room/public maps, the code clears incidentLayer and rescueLayer, fetches new data, then creates fresh L.marker(...) objects at the new coordinates.
- Leaflet popups are rebound with the newest status text and location name.
- If an assigned incident is resolved or missing, the incident marker is removed from the rescue screen.

8. Why this works without WebSockets

WebSockets are used when the server must push updates instantly to clients. Your project is not doing that. Instead, it uses ordinary HTTP requests. That means the server only responds when the frontend asks. Since the frontend asks every few seconds, the data looks live enough. So the correct explanation is: not real-time push, but near-live polling.

9. Strengths and limits of your current approach

Strengths	Limits
Simple to implement with plain JavaScript + Spring Boot.	Not true server push; updates wait for the next polling cycle.
No WebSocket infrastructure needed.	Repeated HTTP calls create extra network/database traffic.
Latest row + history log is a clean database design.	Fast movement can look jumpy because markers refresh in steps.
Good enough for demos, practical exams and moderate refresh needs.	Accuracy still depends on browser/device GPS quality and permissions.

10. Short viva answer you can say

"In NDRCS, live location is implemented using browser geolocation plus polling. The rescue browser gets GPS coordinates after permission, sends them to the Spring Boot backend through POST /api/rescue/update-location, the backend stores the latest position in rescue_team_live_status and also keeps history in rescue_team_location_logs, and the control room/public maps call GET /api/rescue/live-status every 5 seconds to redraw Leaflet markers on OpenStreetMap. So we are not using WebSockets; we are using repeated REST calls."

11. Important file references from your codebase

- FRONTEND/js/rescue-dashboard.js -> asks browser for geolocation, starts 5-second tracking loop, transmits coordinates, updates rescue-side marker and syncs assigned incident marker.
- FRONTEND/js/control-dashboard.js -> polls dashboard, incidents and rescue live status, then redraws control-room markers.
- FRONTEND/js/map.js -> public monitoring map using Leaflet, OpenStreetMap and 5-second polling.
- FRONTEND/js/api.js -> wraps fetch() for GET/POST/PUT/DELETE with base URL <http://localhost:8080>.
- BACKEND/.../controller/RescueTeamController.java -> exposes POST /api/rescue/update-location and GET /api/rescue/live-status.
- BACKEND/.../service/RescueTeamService.java -> validates team account, updates latest live-status row and writes location history log.
- BACKEND/.../model/RescueTeamLiveStatus.java -> current/latest position table.
- BACKEND/.../model/RescueTeamLocationLog.java -> history table for old positions.

Final takeaway: Your project already has a valid live-tracking architecture for a student emergency-response system. The key idea is repeated GPS reading on the rescue side plus repeated backend polling on the monitoring side.